DECA Lab Logbook — Spring Term

Lab 1 | Task 2: Immediate Value MOV Instructions

Task 2: Immediate Value MOV Instructions

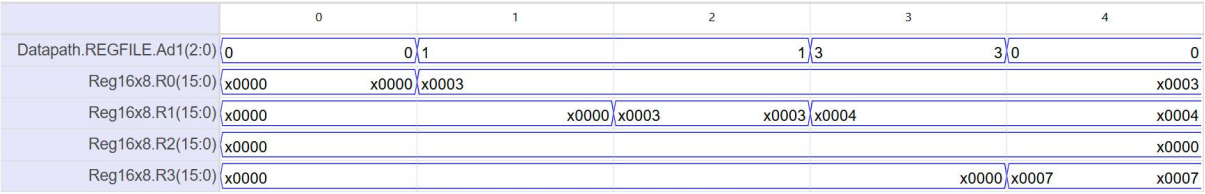
Objectives

- How immediate values are encoded inside the 16-bit instruction word of MOV instructions
- What additional hardware is needed in the datapath to support immediate values

```
MOV R0, #3
MOV R1, R0
ADD R1, #1
ADD R3, R0, R1
```

1: The write address of immediate MOV instructions is still controlled by field a. However, for MOV Ra, #IMM instructions, the data that enters the register file through the data input port comes from the immediate value encoded in the instruction word, not from a register. Run a simulation and use the waveform viewer to confirm that, for both instructions, the correct register is written and the value written is the immediate encoded in the instruction, after sign-extension to 16 bits.

Simulation confirms: MOV R0, #3 writes 0x0003 to R0; MOV R1, R0 copies 0x0003 into R1; ADD R1, #1 produces 0x0004 in R1; ADD R3, R0, R1 stores 0x0007 in R3. The AD1 values confirm that write addresses remain controlled by field a for immediate MOV instructions, consistent with Task 1.



2: Using ALU Instructions : detail, and ALU Instruction Encoding sheet, determine which bit(s) in the 16-bit instruction word control:

i. Whether a MOV instruction reads from an immediate field (#IMM) or from a register

Bit 8 of the instruction word, INS(8), selects the operand source for ALUOPC = 0–6: INS(8) = 0 reads from a register (Rb); INS(8) = 1 reads from the immediate field (#IMM).

OPCALU	Mnemonic	ALU.Out: INS(8)=0	ALU.Out: INS(8) = 1
0	MOV	Rb	Imm
1	ADD	Ra + Rb	Ra + Imm
2	SUB	Ra - Rb	Ra – Imm
3	ADC	Ra + Rb + C	Ra + Imm + C
4	SBC	Ra – Rb + (C-1)	Ra – Imm + (C-1)
5	AND	Ra & Rb	Ra & Imm
6	CMP	Ra - Rb	Ra - Imm

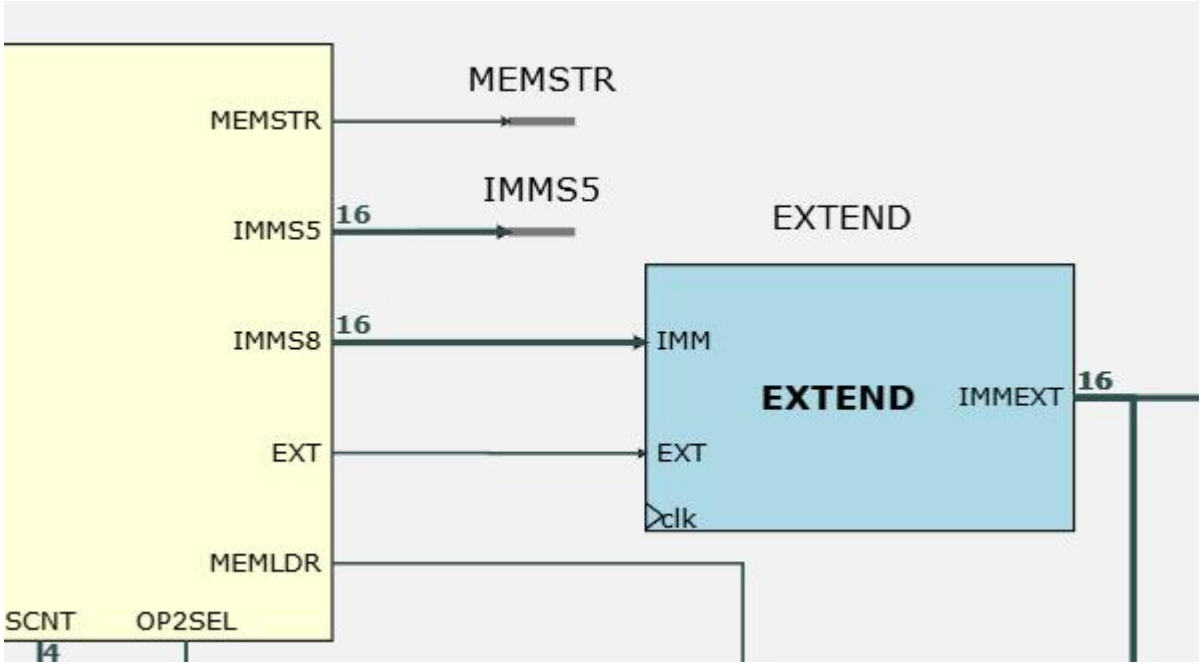
ii. Which bits form the 8-bit immediate value

The 8-bit signed immediate is encoded in bits[7:0] of the instruction word (the Imms8 field).

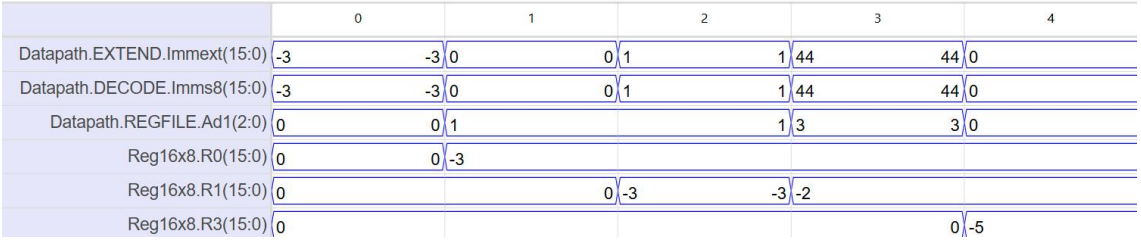
EEP1 machine code	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	ALU	0	ALUOPC=7			a			Shift-opc(1)	b			Shift-opc(0)	Imm4 (SCNT)			
			ALUOPC=0..6			a			0	b			c			(0)	
									1	Imms8 (IMM)							

3: For this question you may assume that, for these MOV instructions, the ALU simply passes the value on its IMM input to the register file's data input port. You may also ignore the EXT block for now; in this lab it behaves as a direct connection with no delay. Open dpdecode and, using your answer to Question ii., investigate through which output port of dpdecode the immediate (#IMM) value is sent in order to reach the ALU's imm port.

The immediate value exits dpdecode through port IMMS8. Inside dpdecode, bits[7:0] are sign-extended to 16 bits and driven onto IMMS8, which connects via the EXTEND block to the ALU's IMM input port.



4: Investigate the hardware that converts the 8-bit immediate into the 16-bit value that travels through the ALU and into the register file. How does this hardware sign-extend the 8-bit field to 16 bits so that negative numbers (for example #-3) are preserved when loaded from the instruction word?

The EXTEND block wires IN(7:0) directly to OUT(7:0) and replicates IN(7) — the sign bit — into each of OUT(15:8). This preserves the two's complement value when widening to 16 bits. For example, #−3 = 0b11111101 (8-bit) becomes 0b1111111111111101 = 0xFFFD (16-bit), which is still −3.

The waveform confirms this: DECODE.Imms8 and EXTEND.Immext both show 0b1111111111111101 when #−3 is loaded, with the upper byte being a direct copy of bit 7.